PYTHON NOTEBOOK ANALYSIS GUIDELINE

A step-by-step learning guide for the 10 business analysis cases

Designed for Jupyter Notebook • Beginner-friendly • No completed business answers or executable solutions • Updated for the full 18-table dataset

How to use this guide

- Work through one case at a time. Treat each numbered step as a notebook section or group of cells. Step numbers restart in every case.
- The guide tells you what to investigate, which DataFrame to use, why it matters, and what to validate. It deliberately does not provide final findings or full code solutions.
- Use the DataFrame names and column names you supplied. The data dictionary uses CSV-style table names; the names in your notebook are lowercase DataFrame variables.
- At each stage, write down the decision you made, any assumptions, and what evidence supports the interpretation.
- Do not treat modeled training data as independently verified company fact. Preserve the Data_Class / Verification_Class distinction in your analysis.

What changed in this version

- production_orders and product_master are now treated as available. Case 6 is rebuilt around production_orders (plan vs actual, delay, efficiency, overtime, bottleneck labels), and product_master is used in Cases 1, 7, 8 and 10.
- customer_service now appears in Cases 2, 3, 7 and 10 as month/channel-level context. It has no key to customers, invoices or products.
- group_operations and business_units now serve as top-down sanity checks in Cases 6 and 10.
- The table list now follows your data dictionary: each table has its grain and its real/modeled status.
- Steps restart at 1 in every case (the earlier version numbered 1 to 94 across the whole guide), and a join map (Appendix D) was added.
- Only Sources remains unavailable. The guide no longer says to substitute proxies for missing tables; proxies are allowed only when labelled.

Data availability and limitations

The guide is based on your data dictionary and the DataFrame names and column lists you supplied. No sample rows or value checks were available, so statements about what a column contains are flagged as things to confirm. The following gaps affect what can be completed:

Limitation	Impact on analysis	Recommended handling
Sources.csv appears in the data dictionary, but no sources DataFrame was supplied.	The audit trail for public facts (product prices, business units, locations) cannot be checked in the notebook.	Rely on Verification_Class in product_master, business_units and locations, and note that source URLs could not be audited.
Notebook DataFrame names are lowercase versions of the dictionary file names.	A name such as Sales_Transactions.csv corresponds to sales_transactions, but the mapping should be confirmed.	Use the actual Python variable names; keep a small mapping note in the notebook.
Most tables are modeled, not real. Only Product_Master, Business_Units and Locations are public/verified; Group_Operations mixes public structure with modeled KPIs.	Metrics are useful for practice but do not represent actual operating performance.	Check Data_Class and Verification_Class values, report them with each result, and do not present modeled results as company fact.
Some tables have no join key between them: customer_service has no Customer_ID or Invoice_ID; production_orders uses Line_ID while printing_batches and machine_downtime use Machine_ID; printing_batches and quality carry no production-order or lot ID.	Linking these at record level would be guesswork and could duplicate or misassign rows.	Join only at product, channel, machine or month level, state the grain of the link, and describe results as associations.
Some measures are not directly represented in the columns: true CAC (no new-customer count), capacity utilization (no available-hours field), standard inventory turns, and any forecast.	These cannot be reported as observed metrics.	Label proxies and assumptions explicitly. Never present a proxy as an observed metric.
Several field meanings are not stated in the dictionary: Reprint_Qty versus Printed_Qty, the Planned flag in machine_downtime, Efficiency_Pct, Delay_Days sign convention, Returned and Quality_Related_Return encoding, and Public_Capacity.	Rates and totals built on a wrong assumption would be misleading.	Inspect each field's values and reconcile quantities first. Write the definition you chose in the notebook.
Join keys are suggested by matching column names, but uniqueness and referential integrity have not been tested.	Joining tables can duplicate rows or silently drop records if keys are not unique or values do not match.	Before any merge, check key uniqueness, nulls, unmatched keys, and row counts before/after the join.

Notebook setup and reusable workflow

Create a separate notebook or clearly separated notebook headings for each case. A consistent sequence makes the work easier to review.

1. **State the question —** Rewrite the business problem as one or two questions that can be answered with measurable evidence.
2. **Identify the unit of analysis —** Determine what one row represents (invoice line, customer, daily snapshot, campaign, inspection, etc.). This prevents accidental double counting.
3. **Inspect the data —** Check shape, columns, data types, date ranges, missing values, duplicates, category labels, and Data_Class/Verification_Class.
4. **Define metrics before calculating —** Write each metric in plain language, including numerator, denominator, time window, and treatment of returns or missing values.
5. **Prepare and validate —** Convert dates and numeric fields appropriately; check impossible values; reconcile totals; retain a raw copy.
6. **Aggregate at the right grain —** Group at the grain needed for the question, then compare like with like. Avoid mixing daily, monthly, campaign, and transaction grains without aggregation.
7. **Visualize and interpret —** Choose charts that expose trends, differences, distributions, or concentration. Explain what the chart shows and what it cannot prove.
8. **Turn evidence into actions —** Connect each proposed action to a finding, quantify the expected direction where possible, and state confidence/limitations.

Table reference

Each table with its grain and real/modeled status from the data dictionary, what to use it for, and the cases that use it.

Notebook DataFrame	Grain and status	Use it for	Key fields	Cases
sales_transactions	1 row per invoice line. Modeled, anchored to public products/prices.	Revenue, profit, returns, channel, product and customer purchase behavior	Invoice_ID, Order_Date, Month_Key, Product_ID, Location_ID, Customer_ID, Quantity, Net_Sales_EGP, Gross_Profit_EGP, Returned, Refund_Value_EGP, Data_Class	1, 2, 3, 4, 5, 7, 10
product_master	1 row per public product. Public/verified.	Public product attributes, season and retail price	Product_ID, SKU, Brand, Product_Family, Gender, Audience, Catalog_Season, Sizes, Public_Retail_Price_EGP, Verification_Class	1, 4, 5, 7, 8, 10
customers	1 row per anonymized customer. Modeled.	Segmentation, cohorts and retention enrichment	Customer_ID, Customer_Type, Governorate, Age_Band, Gender, Join_Date, Acquisition_Channel, Consent_Status	2, 10
locations	1 row per channel/operating location (retail, online, wholesale, HQ, factory). Public where available.	Location, channel and geographic lookup	Location_ID, Location_Name, Location_Type, Channel, Governorate, City, Verification_Class	1, 3, 4, 10
inventory_history	Month x location x product. Modeled; no forecast outputs.	Stock position, coverage, reorder thresholds, stockout history	Snapshot_Month, Location_ID, Product_ID, Opening_Stock, Sold_Qty, Closing_Stock, Days_of_Cover, Stockout_Days_In_Month, Supplier_Lead_Time_Days	4, 9, 10
marketing	1 row per campaign. Modeled.	Spend, engagement, attributed orders and revenue	Campaign_ID, Product_ID, Platform, Start_Date, End_Date, Spend_EGP, Clicks, Orders_Attributed, Attributed_Revenue_EGP, Discount_Pct	5, 10
web_traffic_daily	1 row per day. Modeled.	Daily site funnel and conversion	Date, Sessions, Users, Add_To_Cart_Rate, Checkout_Rate, Orders, Conversion_Rate, Avg_Page_Load_Sec	3, 5, 10
materials	1 row per material. Modeled.	Material attributes, units, category, standard lead time	Material_ID, Unit, Material_Category, Standard_Lead_Time_Days	9, 10
product_bom	Product x material. Modeled.	Material requirements per product	Product_ID, Material_ID, Required_Per_Unit, Unit	9, 10
suppliers	1 row per anonymized supplier. Modeled.	Supplier attributes and contract status	Supplier_ID, Primary_Category, Governorate, Payment_Terms_Days, Contracted	9, 10
purchase_orders	1 row per material PO. Modeled.	Lead time, on-time delivery, cost and rejects	PO_ID, Supplier_ID, Material_ID, Order_Date, Promised_Date, Actual_Receipt_Date, Ordered_Qty, Accepted_Qty, Rejected_Qty, Unit_Cost_EGP	9, 10
production_orders	1 row per production order. Modeled using public capacity context.	Throughput, plan vs actual, efficiency, delay, bottleneck, overtime	Production_Order_ID, Product_ID, Start_Date, Due_Date, Completion_Date, Planned_Qty, Produced_Qty, Good_Qty, Rejected_Qty, Line_ID, Efficiency_Pct, Delay_Days, Primary_Bottleneck, Overtime_Hours	6, 8, 9, 10
printing_batches	1 row per print batch. Modeled.	Yield, defects, reprints, consumable and rework cost	Print_Batch_ID, Product_ID, Print_Date, Print_Type, Machine_ID, Planned_Qty, Printed_Qty, Good_Qty, Rejected_Qty, Reprint_Qty, Defect_Type	6, 7, 8, 10

Notebook DataFrame	Grain and status	Use it for	Key fields	Cases
machine_downtime	1 row per downtime event. Modeled.	Downtime, planned vs unplanned, root reason	Downtime_ID, Date, Machine_ID, Business_Area, Downtime_Hours, Reason, Planned	6, 8, 10
quality	1 row per QC inspection. Modeled.	Defect Pareto, root causes, quality cost	QC_ID, Inspection_Date, Product_ID, Stage, Inspected_Qty, Passed_Qty, Rejected_Qty, Defect_Type, Root_Cause, Estimated_Loss_EGP	6, 7, 8, 10
customer_service	1 row per support ticket. Modeled.	Service levels, issue mix, satisfaction	Ticket_ID, Open_Date, Channel, Issue_Type, First_Response_Minutes, Resolution_Hours, Resolved, CSAT_1_to_5	2, 3, 7, 10
group_operations	Month x business unit. Public unit structure + modeled KPIs.	Monthly cross-business-unit operating KPIs	Month, Business_Unit, Primary_KPI, Volume, Operational_Efficiency_Pct, Open_Issues_Count	6, 10
business_units	1 row per business unit. Public/verified.	Group structure and public capacity facts	Business_Unit, Function, Public_Capacity, Source_URL, Verification_Class	6, 10
sources (not supplied)	1 row per public source. Public.	Audit trail for public facts. Not available as a DataFrame.	Not available. Do not assume columns.	10 (noted as missing)

Case 1. Executive Sales & Profitability

Business question: Is revenue growth translating into healthy gross profit across products and channels?

Target output: An executive view of sales, units, gross profit, margin, growth, seasonality, product contribution and discount impact.

Tables to use

- Primary: sales_transactions (1 row per invoice line; modeled transactions anchored to public products and prices).
- Enrichment: product_master for Brand, Product_Family, Gender, Audience, Catalog_Season and Public_Retail_Price_EGP; locations for location/channel context; customers only if customer attributes matter.

Step-by-step workflow

1. Define the reporting period: Inspect Order_Date and Month_Key coverage. Decide the analysis window and whether enough prior-period data exists for month-over-month (MoM) or year-over-year (YoY) comparisons.
2. Validate the sales grain: Confirm that a row is an invoice line. Check Invoice_ID duplicates and distinguish invoice-line rows from unique invoices; do not count every row as an order.
3. Audit monetary fields: Compare Gross_Sales_EGP, Net_Sales_EGP, Estimated_COGS_EGP, Gross_Profit_EGP and Gross_Margin_Pct for consistency. Establish whether returns/refunds are already reflected in net sales or profit before you use Refund_Value_EGP.
4. Check price integrity against the catalog: Join product_master on Product_ID (or SKU) after checking key uniqueness. Compare List_Price_EGP with Public_Retail_Price_EGP and investigate products where they differ. Keep the public price as a reference, not as proof the transaction is real.
5. Create the core KPI set: Calculate revenue, quantity, gross profit and weighted gross margin by month, product family, SKU, channel and location. Use summed profit divided by summed revenue for a weighted margin rather than averaging row-level percentages.
6. Measure growth and seasonality: Aggregate to month first. Compare each month with the prior month and, where available, the same month in the prior year. Use Catalog_Season from product_master as a second lens on seasonal patterns. Note incomplete periods and changing transaction coverage.
7. Assess product and channel contribution: Rank product families/SKUs, brands, audiences and channels by net sales and gross profit. Compare contribution share, margin and unit volume rather than ranking on revenue alone.
8. Investigate discount impact: Compare Discount_Pct bands with net sales, gross profit and margin. Check whether product mix or channel mix differs across discount bands before implying discounts caused a change.
9. Build executive visuals: Use a time-series view for sales/profit, a ranked bar chart for product/channel contribution, and a margin-versus-revenue comparison. Keep units and currency labels clear.
10. Write the management interpretation: Identify where growth is profitable or unprofitable, what evidence supports the claim, what further check is needed, and which action should be tested.

Validation checklist

- Reconcile totals against a direct aggregation of the same source rows.
- Check negative values, zero revenue, extreme discounts, and return treatment.
- Report weighted margin; state how returns and incomplete months are handled.
- Confirm row count and sales totals before and after the product_master join.
- Separate modeled rows from public/verified facts using Data_Class and Verification_Class.

Case 2. Customer 360 & Retention

Business question: Which customers return, how valuable are they, and where might retention improve?

Target output: Customer segments, repeat rate, retention/cohorts, average order value, frequency, high-value profiles, retention opportunities and service context.

Tables to use

- Primary: sales_transactions.
- Enrichment: customers (1 row per anonymized customer) for Join_Date, Customer_Type, Governorate, Age_Band, Gender, Acquisition_Channel and Consent_Status.
- Context only: customer_service (1 row per ticket). It has no Customer_ID or Invoice_ID, so it cannot be tied to individual customers.

Step-by-step workflow

1. Confirm Customer_ID coverage and how guest/anonymous purchases are represented. Decide whether returns are excluded, netted, or reported separately for purchase behavior.
2. Check that customer attributes agree across tables: sales_transactions carries Customer_Type and Customer_Governorate, and customers carries Customer_Type and Governorate. Test whether they match for the same Customer_ID and decide which source is authoritative.
3. Define the order grain. If one invoice has multiple product lines, aggregate invoice lines to customer-invoice before calculating order count or average order value.
4. Build a customer-level summary using a consistent analysis date: recency (days since last purchase), frequency (distinct invoices/orders), and monetary value (net sales or gross profit, with the chosen return treatment).
5. Create RFM segments by ranking or quantiling recency, frequency and monetary value. Explain the cutoffs; inspect segment sizes so tiny or tied groups do not mislead.
6. Calculate repeat-purchase rate using a clearly defined denominator (for example, identifiable customers with at least two distinct orders divided by identifiable purchasing customers).
7. Calculate average order value from order-level totals, not invoice-line averages. Also examine purchase frequency and gross profit per customer.
8. Build cohorts using customers.Join_Date or first observed purchase date. Check how far apart the two are, state which definition you use, and compare retention by elapsed month since cohort entry.
9. Profile high-value and at-risk segments by customer type, governorate, age band, gender or acquisition channel. Report these at group level only and avoid treating demographic association as a cause.
10. Add service context at the channel and month level: Use customer_service (Issue_Type, First_Response_Minutes, Resolution_Hours, Resolved, CSAT_1_to_5) to see whether service quality moves alongside repeat purchasing in the same channels and periods. Describe it as a parallel trend, not a customer-level effect.
11. Translate findings into testable retention actions, such as targeted win-back or loyalty tests. Define a success metric and a comparison group before recommending rollout.

Validation checklist

- Check that customer-invoice aggregation removes multiple line items from order counts.
- Do not interpret missing Customer_ID as a single real customer.
- Ensure cohort months have comparable follow-up time.
- Use Consent_Status appropriately and do not expose identifiable customer-level outputs.
- Do not link customer_service to customers; there is no shared key.

Case 3. Branch & Channel Performance

Business question: Which sales channels and locations create value rather than just volume?

Target output: Channel growth, margin, returns, basket size, geographic demand, service experience and contribution opportunities.

Tables to use

- Primary: sales_transactions.
- Lookup: locations (1 row per channel/operating location), joined using Location_ID after validating key uniqueness. Customer geography may be used as a separate view, not confused with store location.
- Channel context: web_traffic_daily for the online channel funnel; customer_service by Channel for service experience.

Step-by-step workflow

1. Check how Channel, Location_ID and Location_Name are populated. Identify whether online orders are assigned a location and whether channel labels are consistent across sales_transactions, locations and customer_service.
2. Aggregate sales_transactions by month and Channel to compare net sales, gross profit, weighted margin, quantity, distinct invoices and returns.
3. Aggregate by Location_ID and then join locations for Location_Type, Governorate and City. locations also holds HQ and factory rows; confirm which Location_Type values actually appear in sales before comparing locations. Compare row counts and unmatched keys before and after the join.
4. Calculate basket size at invoice level: total invoice value and quantity, then summarize by channel/location. Do not calculate average basket from individual product lines.
5. Define return rate explicitly (for example, returned invoice lines / invoice lines, or returned quantity / sold quantity). Choose a definition aligned with the question and state it.
6. Compare geographic demand using sales and profit alongside customer or population context only if a reliable denominator is available. Without a denominator, call it observed sales concentration, not market penetration.
7. Add the online funnel: For the online channel, view Sessions, Add_To_Cart_Rate, Checkout_Rate and Conversion_Rate from web_traffic_daily by month next to online sales. Treat it as context; the two tables share no key.
8. Add service by channel: Compare ticket volume, First_Response_Minutes, Resolution_Hours, Resolved rate and CSAT_1_to_5 across channels. Check ticket counts before comparing averages.
9. Separate changes caused by sales volume from changes caused by margin, returns or channel mix. Compare similar periods.
10. Create a ranked view and a channel/location comparison chart; identify underperformers for investigation rather than assuming the location itself caused the result.
11. Write actions tied to evidence, such as assortment, staffing, fulfillment or channel-mix tests, and specify how outcomes would be measured.

Validation checklist

- Check for duplicated Location_ID rows in locations before merging.
- Compare pre- and post-merge transaction counts and sales totals.
- Keep customer Governorate distinct from store/location Governorate.
- Document the selected return-rate denominator.
- Do not attribute web traffic or service tickets to specific locations unless a valid key exists.

Case 4. Inventory & Demand Planning

Business question: Which SKUs are overstocked or at risk of stockout, and how should replenishment be planned?

Target output: ABC/XYZ analysis, stock turns, days of cover, stockout/overstock diagnosis and a self-built 30/60/90-day forecast.

Tables to use

- Primary: inventory_history (month x location x product; modeled, with no forecast outputs by design).
- Demand cross-check: sales_transactions.
- Context: product_master for product attributes; locations for location detail; purchase_orders, product_bom and materials only if you extend the analysis to supply risk (see Case 9).

Step-by-step workflow

1. Inspect Snapshot_Month coverage and confirm that each row represents one month x location x product. Check duplicate keys at that grain.
2. Reconcile stock movement: compare Opening_Stock + Received_Qty + Returned_Qty - Sold_Qty - Damaged_Qty with Closing_Stock. Investigate differences rather than forcing equality.
3. Cross-check demand: Aggregate sales_transactions Quantity to month x location x product and compare with Sold_Qty. Note that sales has a Size field and inventory does not, and that Month_Key and Snapshot_Month may use different formats. Decide how returns are treated on each side.
4. Assess coverage and risk using Days_of_Cover, Avg_Daily_Sales_90D, Safety_Stock_Units, Reorder_Point_Units, Supplier_Lead_Time_Days and Stockout_Days_In_Month. Confirm the units and time basis for each field, and test whether Days_of_Cover is consistent with closing stock and average daily sales.
5. Calculate inventory turns only after choosing a defensible definition and time period. If cost of goods sold is unavailable at the matching inventory grain, state the limitation and use a clearly labelled proxy rather than calling it standard inventory turns.
6. Perform ABC classification by SKU using a chosen value measure (for example, sales value or gross profit over a defined period). State the measure and cumulative-share thresholds used.
7. Perform XYZ classification using demand variability over a sufficiently long, regular time series. Explain how zero-sales months, short histories and intermittent demand are treated.
8. Diagnose stockouts and overstock by combining recent demand, days of cover, reorder point, lead time and stockout days. Segment by location as well as SKU where useful.
9. Build a baseline demand forecast from historical sales at the SKU/month or SKU/location/month grain. Start with a simple benchmark (such as recent average or seasonal-naive forecast) before considering more complex methods.
10. Create separate 30-, 60- and 90-day demand estimates. Convert daily/monthly demand to the matching forecast horizon carefully and align the forecast frequency with the inventory decision.
11. Backtest the forecast using a time-based holdout: train only on earlier dates, predict a later period, and compare forecast with actual demand. Choose error metrics that handle zero/low demand appropriately.
12. Translate forecast and lead-time risk into replenishment logic. Document assumptions for review period, supplier lead time, safety stock and target service level; do not imply these were supplied if they were assumed.

Validation checklist

- No forecast output is supplied in inventory_history; the forecast must be built and evaluated by you.
- Avoid random train/test splits for time series.

- Check stock-balance discrepancies and unit consistency.
- Report forecast uncertainty and flag sparse or short histories.
- Inventory is modeled data: describe results as practice findings, not real operating performance.

Case 5. Marketing Effectiveness

Business question: Which campaigns and platforms appear to generate efficient attributed outcomes?

Target output: ROAS, cost per attributed order and conversion metrics, campaign outliers and evidence-based budget reallocation candidates.

Tables to use

- Primary: marketing (1 row per campaign; modeled).
- Cross-check: web_traffic_daily for site-level funnel context; sales_transactions for observed sales context, with careful attribution limitations.
- Context: product_master for product attributes of the campaign's Product_ID.

Step-by-step workflow

1. Inspect campaign date ranges, platform names, product IDs, spend and attributed outcome fields. Check whether campaigns overlap and whether spend is recorded for comparable periods.
2. Define each metric before calculation: ROAS = attributed revenue / spend; CTR = clicks / impressions; landing-session rate = landing sessions / clicks; attributed order conversion = attributed orders / landing sessions.
3. Decide what to do about CAC: marketing has Orders_Attributed but no count of new customers. Use cost per attributed order and label it as a proxy, or state that true CAC is not calculable.
4. Handle zero denominators and missing values explicitly. A metric should be missing/not defined rather than infinite when its denominator is zero.
5. Compare campaigns by platform, product and period. Include spend and volume next to rates so small campaigns with volatile rates are not overvalued.
6. Inspect Discount_Pct and campaign duration (Start_Date to End_Date). Compare like-for-like campaigns where possible and treat associations as descriptive, not causal.
7. Use web_traffic_daily to understand broad site trends, but do not assign daily traffic or orders to a campaign unless a valid linkage exists.
8. Use sales_transactions as a reality check for overall product/channel movement during campaign windows. Do not assume all observed sales were caused by a campaign or that attributed revenue is incremental revenue.
9. Investigate outliers and tracking anomalies before excluding them. Examples: attributed orders above landing sessions, or attributed revenue far above what sales_transactions shows for that product and window. Keep a note of any filters and their rationale.
10. Rank candidates for budget tests using efficiency, scale, consistency and data quality together. Propose a controlled test and define its success metric.

Validation checklist

- ROAS and cost per order are not interchangeable.
- Confirm the currency and time window of spend and revenue align.
- Do not add attributed revenue to sales_transactions revenue as if they were independent sales.
- Avoid causal claims without a holdout, experiment or credible comparison design.

Case 6. Factory Production & Bottlenecks

Business question: Where does production underperform, and which constraints should be investigated first?

Target output: Throughput, plan-versus-actual, rejects, delay, overtime, bottlenecks and a capacity view, with proxies clearly labelled.

Tables to use

- Primary: production_orders (1 row per production order; modeled using public capacity context).
- Supporting: printing_batches, machine_downtime, quality, business_units (public capacity facts) and group_operations (monthly business-unit KPIs).

Step-by-step workflow

1. Inspect production_orders: shape, date ranges (Start_Date, Due_Date, Completion_Date), missing values, duplicate Production_Order_ID values, Line_ID values and Data_Class. Confirm that one row is one production order.
2. Reconcile quantities: Check whether Produced_Qty equals Good_Qty + Rejected_Qty, and compare Planned_Qty with Produced_Qty. Investigate gaps (overproduction, partial orders, open orders) before computing rates.
3. Define throughput and plan attainment using comparable units and a consistent period. Decide how open or incomplete orders (missing Completion_Date) are treated and report them separately.
4. Verify delay: Recompute delay from Completion_Date and Due_Date and compare with Delay_Days. Check how early completions and missing dates are encoded, and decide whether to analyse delay rate, average delay or both.
5. Examine Efficiency_Pct: confirm its definition and range, and weight it by volume when you aggregate rather than averaging order-level percentages. Treat it as a modeled indicator, not measured OEE.
6. Analyse Primary_Bottleneck and Overtime_Hours: tabulate bottleneck labels by Line_ID, product and period, and compare overtime with delay and efficiency. Check missing or catch-all labels before ranking causes.
7. Capacity utilization: No capacity denominator in hours is provided. business_units.Public_Capacity is context and may not be an available-hours figure, so verify its definition. If you build a utilization measure from Produced_Qty or Efficiency_Pct, label it a proxy.
8. Use printing_batches for the printing view: output, good quantity, rejects and reprints. Treat it as one operation within production, not a replacement for all production orders, and do not label print-batch throughput as total factory throughput.
9. Use machine_downtime to quantify planned and unplanned downtime by machine, Business_Area, reason and period. Verify how Planned is encoded and whether event overlaps exist. production_orders uses Line_ID while downtime and printing use Machine_ID, and no mapping between them is supplied, so do not join them without documenting an assumed mapping.
10. Use quality to compare rejection patterns by stage and root cause. Check that inspection quantities do not overlap or duplicate the same production units, and compare Rejected_Qty in production_orders with rejects in quality at product/period level only.
11. Use group_operations as a top-down sanity check: compare Operational_Efficiency_Pct and Open_Issues_Count for the relevant business unit by month with your bottom-up picture. Do not sum Volume across business units because Primary_KPI differs.
12. Compare patterns across periods or lines, accounting for product mix and operating hours where available. A high delay count alone does not prove a bottleneck.
13. Create a bottleneck hypothesis and list the additional evidence needed to test it. Prioritize actions by estimated impact, confidence and feasibility.

Validation checklist

- Reconcile Planned, Produced, Good and Rejected quantities before calculating any rate.
- Do not label print-batch throughput as total factory throughput.
- Check overlapping downtime events and denominator definitions.
- Label capacity utilization and efficiency measures as proxies unless the definition is confirmed.
- Clearly distinguish measured values from inferred bottleneck hypotheses.

Case 7. Quality, Returns & Cost of Poor Quality

Business question: Which defects and causes contribute most to quality loss and customer returns?

Target output: Defect Pareto, loss by product/stage/root cause, connection to returns and complaints, and corrective-action priorities.

Tables to use

- Primary quality view: quality (1 row per QC inspection).
- Customer outcome view: sales_transactions (Returned, Return_Reason, Refund_Value_EGP, Quality_Related_Return) and customer_service (Issue_Type, CSAT_1_to_5).
- Supporting context: printing_batches (Defect_Type, Rework_Cost_EGP), production_orders (Rejected_Qty) and product_master (Product_Family).

Step-by-step workflow

1. Inspect the grain of quality: one row per QC inspection. Check whether inspected quantities represent unique lots or repeated inspection stages.
2. Calculate inspection pass/reject rates using summed Passed_Qty and Inspected_Qty or a verified reconciliation. Check whether Passed_Qty + Rejected_Qty equals Inspected_Qty.
3. Build Pareto views for Defect_Type and Root_Cause by rejected quantity and Estimated_Loss_EGP. Sort descending and calculate cumulative contribution to identify concentrated causes.
4. Break quality metrics down by product, product family and Stage. Compare rates as well as counts because inspection volume may vary substantially.
5. In sales_transactions, summarize Returned, Return_Reason, Refund_Value_EGP and Quality_Related_Return by product, period and relevant dimensions. Check how Returned and Quality_Related_Return are encoded and how they relate to each other.
6. Add the service signal: Summarize customer_service by Issue_Type and month, with ticket counts, resolution and CSAT_1_to_5. Look for issue types that read like quality problems, and use them as a month-level trend only because tickets cannot be linked to a product or invoice.
7. Connect QC defects to customer returns at a compatible grain, usually product and time period. Use Product_ID and date windows; avoid claiming a particular returned unit came from a particular QC record unless a lot/order linkage exists.
8. Compare defect/return trends over time and identify plausible relationships. Note the lag between production, sale and return.
9. Estimate cost of poor quality using available Estimated_Loss_EGP, Rework_Cost_EGP, Ink_or_Consumable_Cost_EGP and refunds only where definitions do not overlap. Avoid double-counting costs.
10. Prioritize corrective actions by frequency, cost, severity, confidence in root cause and ability to verify improvement. Define a follow-up metric.

Validation checklist

- Confirm inspection quantities reconcile before calculating rates.

- Distinguish returned lines, returned units and refund amounts.
- Product/time matching supports association, not unit-level causation.
- Check whether estimated quality loss already includes rework or scrap cost.
- Do not link customer_service tickets to products or invoices; there is no shared key.

Case 8. Printing Performance & Downtime

Business question: Which machines or print processes lose output through defects, reprints or downtime?

Target output: Machine-level yield, defect/reprint cost, downtime reasons and maintenance priorities.

Tables to use

- Primary: printing_batches (1 row per print batch; modeled).
- Supporting: machine_downtime (1 row per downtime event; shares Machine_ID with printing_batches) and quality.
- Context: production_orders for the wider production picture; product_master for product attributes.

Step-by-step workflow

1. Check Print_Date range, Print_Type, Machine_ID and the grain of a print batch. Printing batches carry no production order ID, so do not assume a batch maps to a production order.
2. Reconcile Planned_Qty, Printed_Qty, Good_Qty and Rejected_Qty. Confirm whether Reprint_Qty is included in Printed_Qty or represents additional work; do not assume these are additive.
3. Define yield using good output divided by an explicitly chosen denominator. Calculate reject rate and reprint rate only after confirming quantity definitions.
4. Summarize Ink_or_Consumable_Cost_EGP and Rework_Cost_EGP by machine, product, print type and period. Normalize cost by good unit where appropriate.
5. Aggregate machine_downtime by Machine_ID, reason, planned/unplanned status and period. Check event overlaps and whether downtime is logged in hours. Check Business_Area so you only use printing-related machines.
6. Compare downtime with batch output and defect rates by machine and time window. Machine_ID is a valid key between the two tables, but a shared machine and date still do not prove a downtime event caused a particular batch's defects.
7. Analyse Defect_Type in printing_batches against Defect_Type and Root_Cause in quality, especially for stages that point to printing problems. Check that the labels are comparable before matching them.
8. Build machine scorecards that include volume, yield, rejects/reprints, cost and downtime; avoid ranking a low-volume machine on raw counts alone.
9. Identify maintenance or process-improvement hypotheses and define a before/after KPI to validate whether the intervention worked.

Validation checklist

- Confirm how reprints are counted in relation to printed quantity.
- Use consistent yield denominators across machines.
- Do not infer machine causation from coincident downtime alone.
- Account for product and print-type mix when comparing machines.

Case 9. Supply Chain & Materials

Business question: Which materials and suppliers present the greatest delivery, quality, cost or shortage risk?

Target output: Supplier/material scorecards, lead-time performance, acceptance rate, cost trends, criticality and an action matrix.

Tables to use

- Primary: purchase_orders (1 row per material PO; modeled).
- Lookup/context: suppliers (anonymized), materials, product_bom (product x material) and inventory_history.
- Demand link: production_orders (Planned_Qty, Produced_Qty by product) and sales_transactions for the volume you need to translate into material requirements.

Step-by-step workflow

1. Inspect PO_ID uniqueness, date fields, material/supplier keys, units, ordered quantities and accepted/rejected quantities.
2. Calculate actual lead time as Actual_Receipt_Date minus Order_Date for received orders. Compare with Standard_Lead_Time_Days from materials and distinguish standard lead time from promised delivery date.
3. Define on-time delivery against Promised_Date. Decide how to treat missing receipt dates, partial receipts and canceled/open POs; report them separately rather than silently dropping them.
4. Calculate accepted quantity rate using accepted quantity divided by the appropriate received/inspected quantity. Check whether Accepted_Qty + Rejected_Qty reconciles to received quantity.
5. Track Unit_Cost_EGP over time by material and supplier. Compare only compatible units/currencies and account for changes in material mix or order size.
6. Join suppliers and materials using Supplier_ID and Material_ID after checking key uniqueness and unmatched rows. Compare supplier performance across cost, lead time, acceptance, Payment_Terms_Days and Contracted status.
7. Use product_bom to map material requirements to products. Estimate material demand from production_orders volume (or sales volume if you choose) multiplied by Required_Per_Unit, check that Unit matches in purchase_orders, materials and product_bom, and label the result an estimate. Document how criticality is assigned.
8. Use inventory_history to identify stockout exposure and compare with supplier lead time and reorder points. Note that inventory is at product level while purchasing is at material level, so connect them only through product_bom. A shortage-risk score should be transparent and rule-based.
9. Create a supplier/material action matrix (for example, high criticality/high risk versus low criticality/low risk). Explain each score component and propose a monitoring cadence.

Validation checklist

- Handle open or unreceived POs separately in lead-time metrics.
- Check that units match between purchase_orders, materials and product_bom.
- Avoid treating promised lead time and actual lead time as the same measure.
- Explain how material criticality and shortage risk are defined.
- Suppliers are anonymized and modeled; do not present scorecards as real supplier assessments.

Case 10. Integrated Management Story + Forecast

Business question: How do market demand, customer behavior, marketing, inventory, supply chain, production and quality fit together?

Target output: A connected management narrative, forecast/scenarios, quantified actions and transparent assumptions.

Tables to use

- Demand and profit: sales_transactions, product_master, web_traffic_daily, marketing, customers.
- Availability and materials: inventory_history, purchase_orders, suppliers, materials, product_bom.
- Execution and quality: production_orders, printing_batches, machine_downtime, quality.
- Service: customer_service (month and channel level only).
- Management context: group_operations, business_units, locations.

Step-by-step workflow

1. Start with a single executive question and a fixed reporting period. Decide which outcomes matter most: profitable demand, availability, delivery reliability, quality or cash tied in stock.
2. Create a data map showing each table's grain, date field, primary key candidate and join keys. Use Appendix D as a starting point. Mark Sources as unavailable and mark each table as public/verified or modeled before merging.
3. Build separate validated summaries for sales/profit, customer behavior, campaign/web funnel, inventory, supplier performance, production, printing/downtime, quality and customer service. Do not merge all raw tables into one giant table.
4. Align summaries to a common time grain (often month) only after each source is aggregated correctly. Document that a monthly summary can hide daily timing and operational lag.
5. Build a demand view from sales_transactions and relevant web/marketing indicators. Treat traffic and campaign attribution as explanatory context unless a credible causal design is available.
6. Connect demand to inventory coverage and stockout days. Connect material availability to purchase-order lead times and BOM requirements where units and keys reconcile.
7. Connect execution to production orders, print output, downtime and quality: plan attainment, delay, efficiency, yield, downtime hours and reject rates by month.
8. Connect quality to customer outcomes: returns, quality-related returns, refund value and the service signal, at product and month level without claiming unit-level links.
9. Cross-check against group_operations: Compare your bottom-up monthly picture with Operational_Efficiency_Pct and Open_Issues_Count by business unit. Explain gaps instead of forcing agreement, and do not add up Volume across units with different Primary_KPI.
10. Create a small number of scenarios (for example, baseline, higher demand, and supply/quality disruption). State every assumption, how it changes the forecast, and which metrics are recalculated.
11. Quantify actions using transparent arithmetic and distinguish observed baseline values from scenario assumptions. Avoid implying scenario outputs are guaranteed outcomes.
12. Present a short story in order: what is happening, where value/risk is concentrated, likely drivers, evidence limits, actions, expected KPI movement and how to monitor.
13. End with a decision register: proposed action, supporting evidence, owner/function, leading indicator, outcome KPI, review date and key dependency.

Validation checklist

- Keep each source at its native grain until summarized.
- Do not double-count marketing-attributed revenue and transaction revenue.
- Do not claim causal links without appropriate linkage or design.
- Mark the missing Sources table and any proxies (utilization, efficiency, cost per attributed order).
- State that most operating tables are modeled data; present results as a practice analysis, not as real company performance.
- Separate actual observations, estimates, and scenario assumptions visually and in prose.

Appendix A. Join planning checklist

Before joining any two DataFrames, answer these questions in your notebook and write down the result.

- What does one row in each table represent?
- What key or combination of keys should link the tables?
- Is the key unique on the lookup side? If not, will the merge multiply rows?
- How many null and unmatched keys exist on each side?
- What are the row count and key business totals before and after the merge?
- Is the date grain aligned, or could the join match one row to many periods/events?
- Does the join preserve the intended unit of analysis?

Appendix B. Metric definition worksheet

Metric	Definition to write before calculation	Validation question
Revenue	Which sales field? Gross or net? Are refunds/returns included?	Does the definition match the case brief and period?
Gross margin	Sum of gross profit divided by sum of the selected revenue measure.	Are numerator and denominator defined consistently?
Repeat rate	Customers with at least two distinct orders divided by eligible identifiable customers.	Are orders counted at invoice level?
Return rate	Choose line-, unit-, order- or value-based denominator.	Is the denominator consistent across comparisons?
ROAS	Attributed revenue divided by campaign spend.	Do revenue and spend cover compatible periods and attribution rules?
Cost per attributed order	Spend divided by attributed orders (a proxy; true CAC needs a new-customer count).	Is it labelled as a proxy everywhere it appears?
Yield	Good output divided by a verified input/output denominator.	Are reprints and rejected units counted consistently?
Plan attainment	Produced (or good) quantity divided by planned quantity, for comparable orders and periods.	How are open, partial and over-produced orders handled?
Delay	Completion date versus due date, in days, for completed orders.	Does the recomputed value match Delay_Days, and how are early orders encoded?
On-time delivery	Orders received on or before promised date divided by eligible due/received orders.	How are open, partial and missing-date orders handled?
Service level / CSAT	Resolved share, response and resolution times, and mean CSAT by channel and period.	Are ticket counts large enough for the averages to mean anything?
Forecast error	Choose error metric(s) and evaluation window before comparing models.	Does the metric behave sensibly with zero/low demand?

Appendix C. Final review before submission

- Every chart has a title, period, units and a readable legend.
- Every metric has a written definition and denominator.
- All merges have key and row-count checks.
- No missing table or field is silently replaced by an unsupported proxy; every proxy is labelled.
- Modeled data is clearly distinguished from public/verified facts.
- The narrative separates observed results, interpretation, assumptions and recommendations.

- Each recommendation has a measurable follow-up KPI.
- The notebook can be rerun from top to bottom without relying on hidden manual state.

Appendix D. Join map and link limits

Candidate links between tables, based on matching column names and the dictionary grains. Uniqueness and referential integrity are untested, so run the Appendix A checks before using any of them. A row that says "no key" or "only" is a link you should not make at record level.

Table A	Table B	Key / link	Caution
sales_transactions	locations	Location_ID	Check uniqueness in locations; locations also holds HQ/factory rows.
sales_transactions	customers	Customer_ID	Handle guest/missing IDs; compare duplicated attributes (type, governorate).
sales_transactions	product_master	Product_ID or SKU	Check uniqueness; compare List_Price_EGP with Public_Retail_Price_EGP.
sales_transactions	inventory_history	Product_ID + Location_ID + month	Sales has Size, inventory does not; Month_Key and Snapshot_Month formats may differ.
marketing	product_master / sales_transactions	Product_ID + campaign dates	Attributed revenue is not independent of sales revenue.
marketing	web_traffic_daily	Date range only (no key)	Context only; do not assign daily traffic to a campaign.
printing_batches	machine_downtime	Machine_ID + date	Association only; a shared machine and date do not prove cause.
production_orders	printing_batches / quality	Product_ID + date window	No order or lot ID in the other tables; do not claim record-level links.
production_orders	machine_downtime	No key (Line_ID vs Machine_ID)	Needs an assumed mapping; document it or avoid the link.
quality	sales_transactions	Product_ID + time window	Association with a lag; no unit-level causation.
customer_service	sales_transactions / web_traffic_daily	Channel + month only	No Customer_ID, Invoice_ID or Product_ID in tickets.
purchase_orders	suppliers	Supplier_ID	Check uniqueness and unmatched keys.
purchase_orders	materials	Material_ID	Check that units match.
materials	product_bom	Material_ID	Check Unit consistency with purchase_orders.
product_bom	product_master / production_orders	Product_ID	Material demand is an estimate (volume x Required_Per_Unit).
group_operations	business_units	Business_Unit	Primary_KPI differs by unit; do not sum Volume across units.